

Frontend Developer — Step 2

MASTER CSS

Detailed Notes: CSS Fundamentals, Layout, Responsive Design, Advanced Styling,
Transitions & Animations

CareerCompass Learning Resource

Beginner-friendly notes with concepts, diagrams, code examples, practical patterns,
debugging guidance and exercises.

Learning Map

HTML gives a webpage structure. CSS controls how that structure looks, sizes, aligns, responds and behaves visually.

HTML → DOM → CSS matching → Cascade/Specificity → Layout → Paint → Screen

1	CSS Fundamentals
2	Selectors, Cascade & Specificity
3	Colors, Typography & Backgrounds
4	Box Model, Display & Overflow
5	Positioning & Stacking
6	Flexbox
7	CSS Grid
8	Responsive Web Design
9	Advanced Styling
10	Pseudo-classes & Pseudo-elements
11	Variables, Functions & Units
12	Transitions & Animations
13	Responsive Mini Project
14	Debugging & Best Practices
15	Practice + Interview Checklist

1. CSS Fundamentals

CSS (Cascading Style Sheets) is the language used to style and lay out HTML documents. CSS can control color, typography, spacing, sizing, positioning, layout and visual effects.

Core syntax

```
selector { property: value; }
```

Example

```
p {  
  color: navy;  
  font-size: 18px;  
  line-height: 1.6;  
}
```

Ways to apply CSS

Inline CSS is written on an element; internal CSS is inside a <style> element; external CSS is stored in a .css file. External CSS is normally preferred for maintainability.

External stylesheet

```
<link rel="stylesheet" href="style.css">
```

How the browser uses CSS

The browser parses HTML into the DOM, matches CSS selectors to elements, resolves conflicts through the cascade and specificity, calculates layout, then paints the result.

Inheritance

Properties such as color and font-family commonly inherit. Margin, padding, border and width do not normally inherit.

2. Selectors, Cascade & Specificity

Common selectors

```
* { box-sizing: border-box; }  
p { color: #333; }  
.card { padding: 20px; }  
#hero { min-height: 400px; }  
input[type="email"] { border: 1px solid #aaa; }  
nav a { text-decoration: none; }  
.card > img { width: 100%; }
```

Combinators

Descendant selects nested elements; > selects direct children; + selects an adjacent sibling; ~ selects following siblings.

Cascade

When several rules match the same element, CSS resolves conflicts using origin/importance, specificity and source order.

Specificity

A practical hierarchy is: inline styles > IDs > classes/attributes/pseudo-classes > elements/pseudo-elements. Avoid excessive IDs and avoid using !important as a normal styling technique.

3. Colors, Typography & Backgrounds

Colors

```
body {
  color: #1f2937;
  background: #f8fafc;
}
.button {
  background: rgb(37 99 235);
  color: white;
}
```

Typography

```
body {
  font-family: Arial, sans-serif;
  font-size: 16px;
  line-height: 1.6;
}
h1 {
  font-size: 2.5rem;
  font-weight: 700;
  letter-spacing: -0.02em;
}
```

Useful properties

font-family, font-size, font-weight, line-height, letter-spacing, text-align, text-decoration and text-transform.

Background images

```
.hero {
  background-image: url("hero.jpg");
  background-size: cover;
  background-position: center;
  background-repeat: no-repeat;
}
```

4. Box Model, Display & Overflow

Box model

Every normal CSS box can be understood as: content → padding → border → margin. Think of it as layers around the content.

Recommended sizing

```
* {
  box-sizing: border-box;
}
```

Example

```
.card {
  width: 320px;
  padding: 20px;
  border: 2px solid #ddd;
  margin: 20px;
}
```

Display

block starts a new line; inline flows with text; inline-block flows inline but accepts width/height; none removes the element from layout; flex and grid activate modern layout systems.

Overflow

Use overflow: visible, hidden, auto or scroll depending on whether content should remain visible, be clipped or become scrollable.

5. Positioning & Stacking

Position values

static is the default; relative keeps normal flow while allowing offsets; absolute is positioned relative to a containing block; fixed is normally relative to the viewport; sticky switches between normal flow and a stuck position within its scrolling context.

Common pattern

```
.card { position: relative; }
.badge {
  position: absolute;
  top: 10px;
  right: 10px;
}
```

z-index

z-index controls stacking order inside stacking contexts. If z-index seems broken, inspect positioning and stacking contexts before simply increasing the number.

6. Flexbox

What is Flexbox?

Flexbox is a one-dimensional layout system. It is ideal for rows or columns of related items such as navbars, button groups, toolbars and card rows.

Mental model

Flex container → main axis + cross axis → flex items.

Core example

```
.container {
  display: flex;
  flex-direction: row;
  justify-content: space-between;
  align-items: center;
  gap: 20px;
  flex-wrap: wrap;
}
```

Core properties

flex-direction chooses the main axis; justify-content distributes items on the main axis; align-items aligns on the cross axis; gap adds consistent spacing; flex-wrap allows multiple lines.

Item sizing

```
.item {
  flex-grow: 1;
  flex-shrink: 1;
  flex-basis: 200px;
}
/* shorthand */
.item { flex: 1 1 200px; }
```

When to use

Use Flexbox when the problem is mainly one-dimensional alignment/distribution. Use Grid when you need a two-dimensional row-and-column structure.

7. CSS Grid

What is Grid?

CSS Grid is a two-dimensional layout system designed for rows and columns. It is excellent for page layouts, dashboards, galleries and responsive card grids.

Basic grid

```
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}
```

Important concepts

grid-template-columns/rows define tracks; fr distributes free space; gap controls spacing; minmax() defines flexible minimum/maximum sizes; auto-fit and auto-fill help create responsive repeating tracks.

Responsive cards

```
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1rem;
}
```

Grid vs Flexbox

Flexbox = one dimension. Grid = two dimensions. Professional layouts commonly combine them.

8. Responsive Web Design

Goal

Responsive design makes an interface adapt to different viewport sizes instead of assuming one fixed screen.

Viewport meta tag

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Mobile-first media queries

```
.cards {
  display: grid;
  grid-template-columns: 1fr;
}
@media (min-width: 768px) {
  .cards { grid-template-columns: repeat(2, 1fr); }
}
@media (min-width: 1100px) {
  .cards { grid-template-columns: repeat(3, 1fr); }
}
```

Responsive rules

Prefer fluid widths and max-width. Use flexible Grid/Flexbox layouts. Make images responsive. Test narrow mobile widths and larger desktop widths. Choose breakpoints where your layout needs them rather than targeting every device model.

9. Advanced Styling

Borders and shadows

```
.card {  
  border: 1px solid #e5e7eb;  
  border-radius: 16px;  
  box-shadow: 0 8px 24px rgb(0 0 0 / 0.08);  
}
```

Gradients

```
.hero {  
  background: linear-gradient(135deg, #111827, #2563eb);  
}
```

Modern sizing functions

```
width: min(100% - 2rem, 1100px);  
font-size: clamp(1.5rem, 4vw, 3rem);  
padding: max(1rem, 2vw);
```

Next-level CSS

After fundamentals, explore logical properties, container queries, CSS nesting and newer layout capabilities while checking browser support when necessary.

10. Pseudo-classes & Pseudo-elements

Pseudo-classes

```
a:hover { text-decoration: underline; }  
button:focus-visible { outline: 3px solid currentColor; }  
input:invalid { border-color: crimson; }  
li:nth-child(2) { font-weight: 700; }
```

Pseudo-elements

```
h2::before { content: "→ "; }  
.card::after {  
  content: "";  
  display: block;  
  height: 3px;  
}
```

Accessibility

Do not use generated content as the only source of meaningful information that should be available in the HTML/accessibility tree.

11. Variables, Functions & Units

Custom properties

```
:root {  
  --primary: #2563eb;  
  --text: #1f2937;  
  --radius: 12px;  
  --space: 1rem;  
}  
.button {  
  background: var(--primary);  
  color: white;  
  border-radius: var(--radius);
```

```
padding: var(--space);
}
```

Units

px is useful for precise values; % is relative to a containing dimension; rem is relative to the root font size; em is context-relative; vw/vh are viewport-relative; fr is used by Grid.

Rule of thumb

Choose units according to the behavior you want. Do not force every property to use the same unit.

12. Transitions & Animations

Transition example

```
.button {
  background: #2563eb;
  transform: translateY(0);
  transition: background-color 180ms ease, transform 180ms ease;
}
.button:hover { transform: translateY(-2px); }
```

Keyframes

```
@keyframes slide-in {
  from { opacity: 0; transform: translateY(20px); }
  to { opacity: 1; transform: translateY(0); }
}
.hero-card {
  animation: slide-in 500ms ease-out both;
}
```

Reduced motion

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
    scroll-behavior: auto !important;
  }
}
```

Difference

A transition describes a change between states. A keyframe animation can describe multiple stages over time.

13. Practical Responsive Project

Project

Build a responsive Learning Dashboard containing a header, hero section, course cards, a sidebar on desktop, stacked content on mobile, hover/focus states, Grid, Flexbox, responsive typography and one subtle animation.

HTML structure

```
<main class="page">
  <section class="hero">
    <h1>Learn Frontend Development</h1>
    <p>Build practical skills with structured resources.</p>
    <a class="button" href="#courses">Explore Courses</a>
  </section>
  <section id="courses" class="course-grid">
    <article class="course-card">HTML</article>
    <article class="course-card">CSS</article>
```



```
<article class="course-card">JavaScript</article>
</section>
</main>
```

Layout CSS

```
.page {
  width: min(100% - 2rem, 1100px);
  margin-inline: auto;
}
.course-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1rem;
}
.button {
  display: inline-flex;
  align-items: center;
  justify-content: center;
}
```

14. Debugging & Best Practices

DevTools workflow

Inspect the element → inspect computed styles → identify which rule wins → inspect the box model → test a temporary change → put the final fix into the stylesheet.

Common issues

If a style does not apply, check selector, specificity, stylesheet loading and syntax. If content overflows, inspect fixed widths, long words, images and flex/grid sizing. If z-index fails, inspect stacking contexts. If mobile breaks, inspect fixed widths, viewport settings and media queries.

Best practices

Use meaningful reusable classes; keep CSS organized; use consistent spacing/typography; prefer flexible responsive layouts; preserve keyboard focus visibility; avoid unnecessary animation; optimize images; use DevTools instead of guessing.

15. Practice + Interview Checklist

Practice tasks

Create a centered Flexbox card; build a 3-column responsive Grid; make a wrapping navbar; build a gradient hero with responsive typography; create hover/focus button states; build a pricing section; animate a card entrance with reduced-motion support.

Interview questions

What is CSS? Explain the box model. What is specificity? Difference between display:none and visibility:hidden? Explain relative/absolute/fixed/sticky. Flexbox vs Grid? Main vs cross axis? What is a media query? What is mobile-first? What are custom properties? Transition vs animation? Why use :focus-visible?

Mastery checklist

I can write selectors and understand specificity; explain the cascade; style typography/backgrounds; use box-sizing; use Flexbox; build Grid layouts; create responsive layouts; use pseudo-classes/elements; use variables and modern units; create accessible animations; debug with DevTools; build a responsive page without a CSS framework.

Quick Reference: Flexbox vs Grid

Question	Flexbox	Grid
Dimension	One-dimensional	Two-dimensional
Best for	Rows/columns of related items	Page/section layouts with rows + columns
Key controls	justify-content, align-items, flex	grid-template-columns/rows, gap, areas
Typical examples	Navbar, toolbar, button group	Dashboard, gallery, card grid

Recommended study order: CSS syntax → selectors/cascade → box model → Flexbox → Grid → responsive design → advanced styling → animations → debugging.

Reference: MDN Web Docs CSS documentation is recommended for standards-based property/reference lookup.